

When Zero-Cost Abstractions Aren't Zero-Cost

STEVE SORKIN

When Zero-Cost Abstractions Aren't Zero-Cost

Engineering

Bloomberg

CppCon 2026
September 17, 2026

Steve Sorkin
Senior Software Engineer

TechAtBloomberg.com

About Me

- ❖ At Bloomberg since 2019
- ❖ Primarily a C++ back-end developer
- ❖ Previously a derivatives attorney



Bloomberg

Engineering

Structure of the Talk

- What zero-cost abstractions actually promise
- The assumptions that make them “zero cost”
- What happens when those assumptions break
- Real examples from modern C++
- When abstractions can - and can't - be trusted



What Are Zero-Cost Abstractions?

- Abstractions express *what* not *how*
- High-level, expressive code
- No runtime overhead compared to low-level equivalents
- You don't pay for what you don't use



The Promise

“What you don’t use, you don’t pay for.

What you do use, you couldn’t hand-code any better.”

- Bjarne Stroustrup



Why This Matters

- Modern C++ relies heavily on abstraction
- Performance-sensitive systems depend on this promise
- We often assume high-level code remains efficient



In Practice

- Code looks clean and idiomatic
- Modern abstractions are used correctly
- Performance still falls short of expectations



Zero-Cost Abstractions Are Conditional

- Zero-cost depends on the abstraction being optimized away
- That only works when the compiler can eliminate the overhead
- Otherwise, the abstractions remain part of the runtime

Zero-cost is a contract, not a guarantee





The Contract

Why Zero-Cost Abstractions Can Work

- Abstractions can compile down to efficient low-level code
- The compiler can often eliminate abstraction overhead
- This only works when enough information remains visible



Abstractions Must Remain Transparent

- Concrete types must be visible
- Control flow must be analyzable
- Abstraction boundaries must remain transparent



Some Costs Cannot Disappear

- Allocations introduce real runtime work
- Indirection can remain part of execution
- Dynamic behavior may not be removable





When It Works

The Abstraction Can Disappear Completely

- The abstraction itself imposes no additional runtime overhead
- Only the underlying operation remains



Equivalent Generated Code

```
void transform_version(const std::vector<int>& input,
                      std::vector<int>& output) {
    std::transform(input.begin(), input.end(), output.begin(),
        [](int x) {
            return x * 2;
        });
}
```



```
mov     edi,DWORD PTR [rsi+rax*1]
lea     edx,[rdi+rdi*1]
mov     DWORD PTR [r8+rax*1],edx
add     rax,0x4
cmp     rax,rcx
jne
```

```
void manual_version(const std::vector<int>& input,
                   std::vector<int>& output) {
    for (std::size_t i = 0; i < input.size(); ++i) {
        output[i] = input[i] * 2;
    }
}
```



```
mov     edx,DWORD PTR [rcx+rax*4]
add     edx,edx
mov     DWORD PTR [rsi+rax*4],edx
add     rax,0x1
cmp     rax,rdi
jb
```



When It Breaks

Issues Can Be Hard to Spot

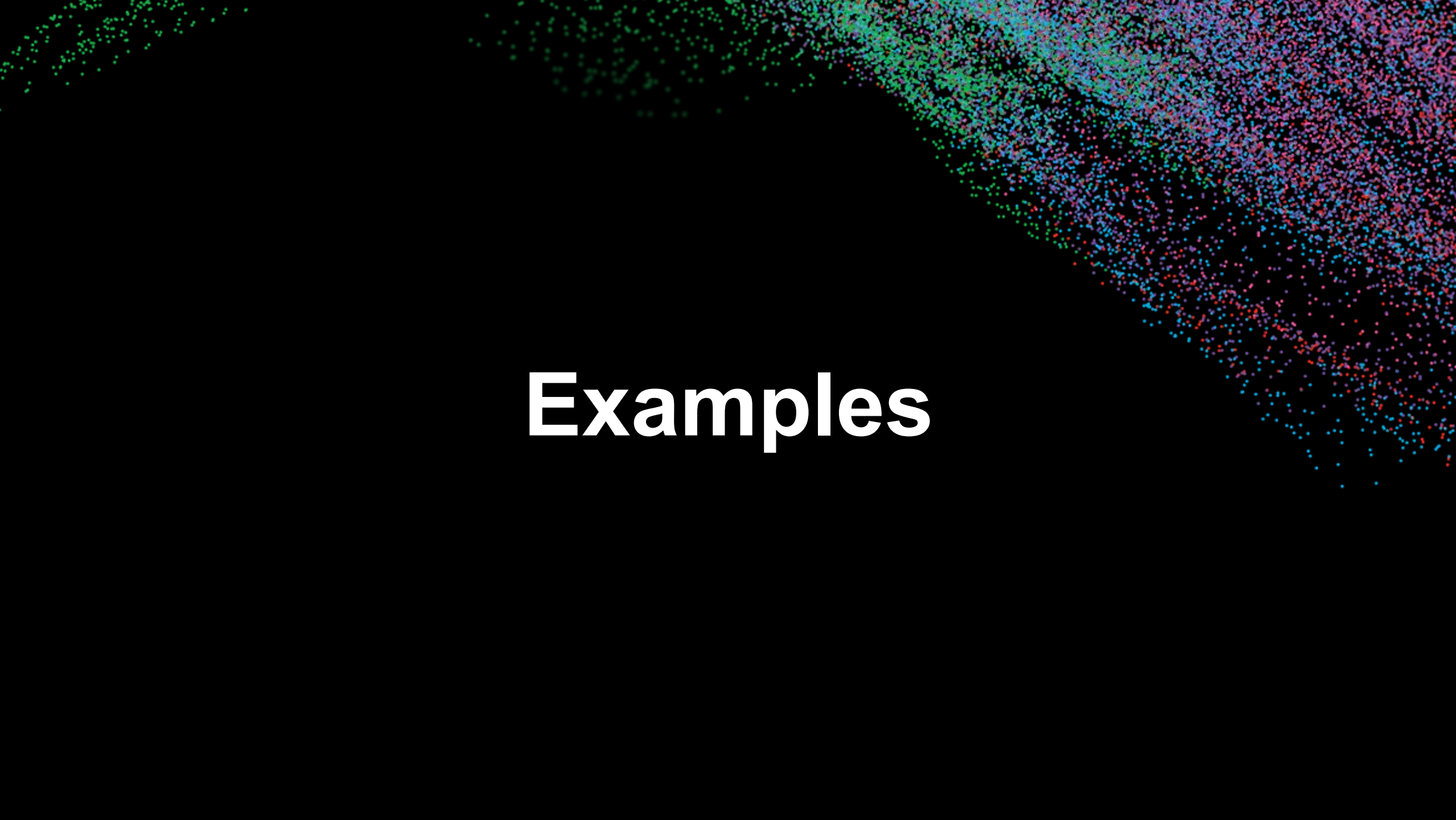
- Clean and idiomatic
- Locally reasonable
- Performance costs are not obvious



Small Abstraction Costs Accumulate

- Small overheads add up over time
- Hot paths amplify abstraction cost
- Costs often appear only at scale





Examples

Configurable Order Processing

- Order processing systems often need configurable behavior
 - ◆ Pricing logic
 - ◆ Risk checks
 - ◆ Validation rules
- Different customers or instruments may require different strategies
- These processing paths may execute millions of times per day



Compile-Time Fee Model Selection

```
struct FixedFeeModel {  
    void operator()(Order& order) const {  
        order.fee = 200;  
    }  
};  
  
struct PercentageFeeModel {  
    void operator()(Order& order) const {  
        order.fee = order.price * 0.001;  
    }  
};
```

```
template <typename FeeModel>  
void process_orders(Orders& orders, FeeModel fee_model)  
{  
    for (auto& order : orders) {  
        fee_model(order);  
    }  
}
```

- Fee model is selected at compile time
- Concrete type remains visible to the compiler
- Calls often can be fully inlined

Runtime Fee Model Selection

```
void process_orders(  
    Orders& orders,  
    std::function<void(Order&)> fee_model) {  
  
    for (auto& order : orders) {  
        fee_model(order);  
    }  
}
```

```
using FeeFn = std::function<void(Order&)>;  
  
FeeFn fee_model =  
    use_fixed_fee ? FeeFn{FixedFeeModel{}}  
                  : FeeFn{PercentageFeeModel{}};
```

- Fee model selected dynamically
- Same processing interface regardless of fee model
- `std::function` hides the concrete callable type

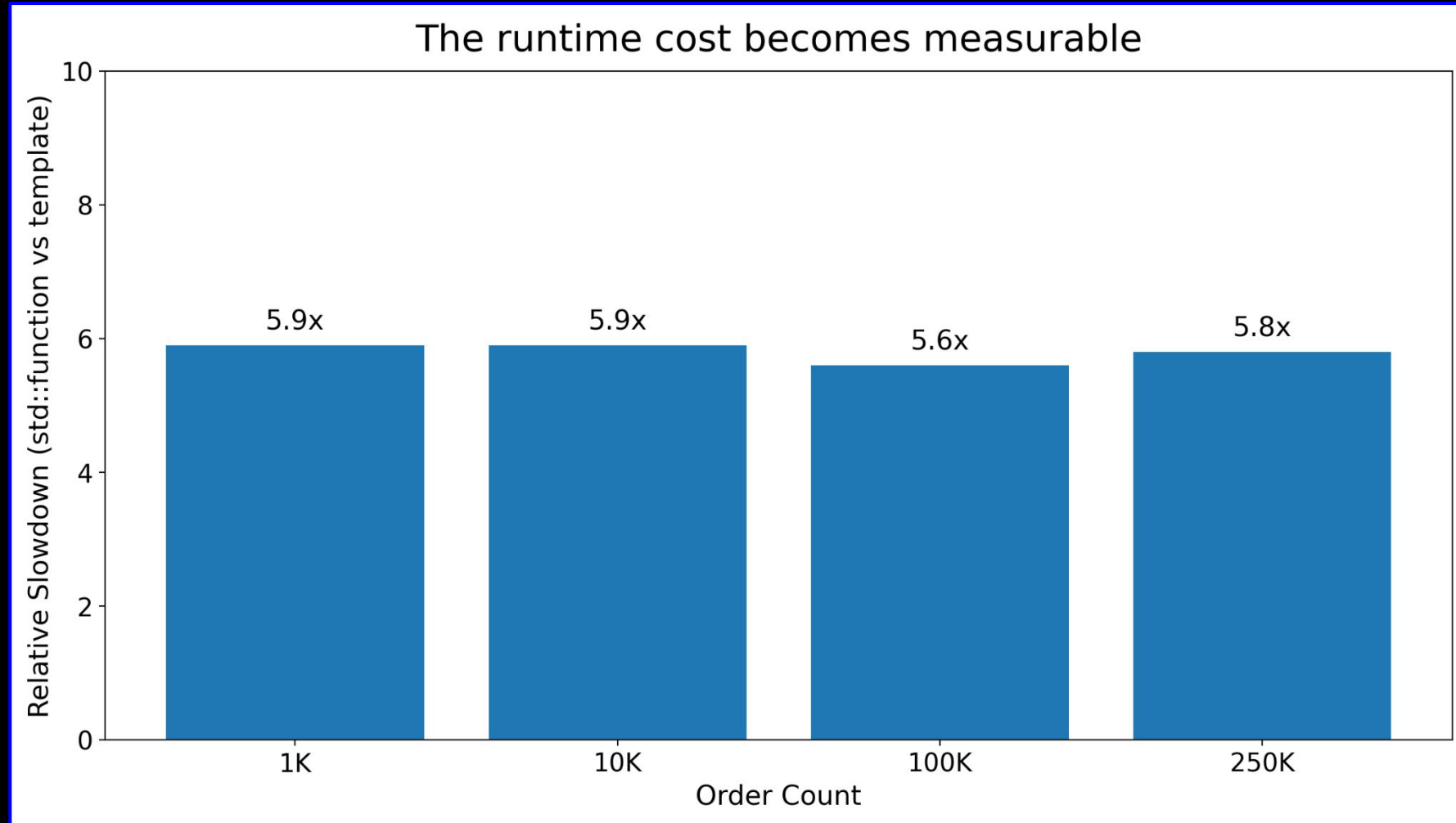
The Abstraction Survives

```
movsd  xmm0, QWORD PTR [rip+0x0]  
...  
movsd  QWORD PTR [rax+0x8],xmm0  
add     rax,0x20  
movsd  QWORD PTR [rax+0x8],xmm0  
cmp     rdx,rax  
jne
```

```
cmp     QWORD PTR [rbp+0x10],0x0  
je      throw_bad_function_call  
mov     rsi,rbx  
add     rbx,0x10  
mov     rdi,rbp  
call    QWORD PTR [rbp+0x18]
```

- Template version: Fee logic is inlined into the loop
- `std::function`: callable target is erased
- Runtime dispatch remains in the generated code

Measuring Abstraction Costs



Processing Market Data Updates

- Trading systems process continuous streams of market data
- Updates are often filtered, transformed and aggregated
- Processing pipelines may run continuously in hot paths
- Readability and composability still matter in performance-sensitive code



Manual Market Data Processing Loop

```
constexpr double min_notional = 1'000'000.0;
constexpr double min_adjusted_price = 100.0;
constexpr double adjustment_factor = 1.01;
double sum = 0.0;

for (const auto& update : updates) {
    if (!update.active)
        continue;
    if (update.price * update.quantity <= min_notional)
        continue;

    double adjusted_price = update.price * adjustment_factor;

    if (adjusted_price <= min_adjusted_price)
        continue;

    sum += adjusted_price;
}
```

- Filtering and transformation handled in one loop
- Branching and data flow remain directly visible to the compiler
- Minimal abstraction layers for the optimizer to analyze

Composable Ranges Pipeline

```
auto prices = updates | std::views::filter([](const Update& u) {  
    return u.active;  
})  
| std::views::filter([](const Update& u) {  
    return u.price * u.quantity > min_notional;  
})  
| std::views::transform([adjustment_factor](const Update& u) {  
    return u.price * adjustment_factor;  
})  
| std::views::filter([](double price) {  
    return price > min_adjusted_price;  
});  
  
double sum = 0.0;  
for (double price : prices) {  
    sum += price;  
}
```

```
constexpr double min_notional = 1'000'000.0;  
constexpr double min_adjusted_price = 100.0;  
constexpr double adjustment_factor = 1.01;
```

- Same processing steps expressed declaratively
- Filters and transformation become separate pipeline stages
- Laziness avoids intermediate containers

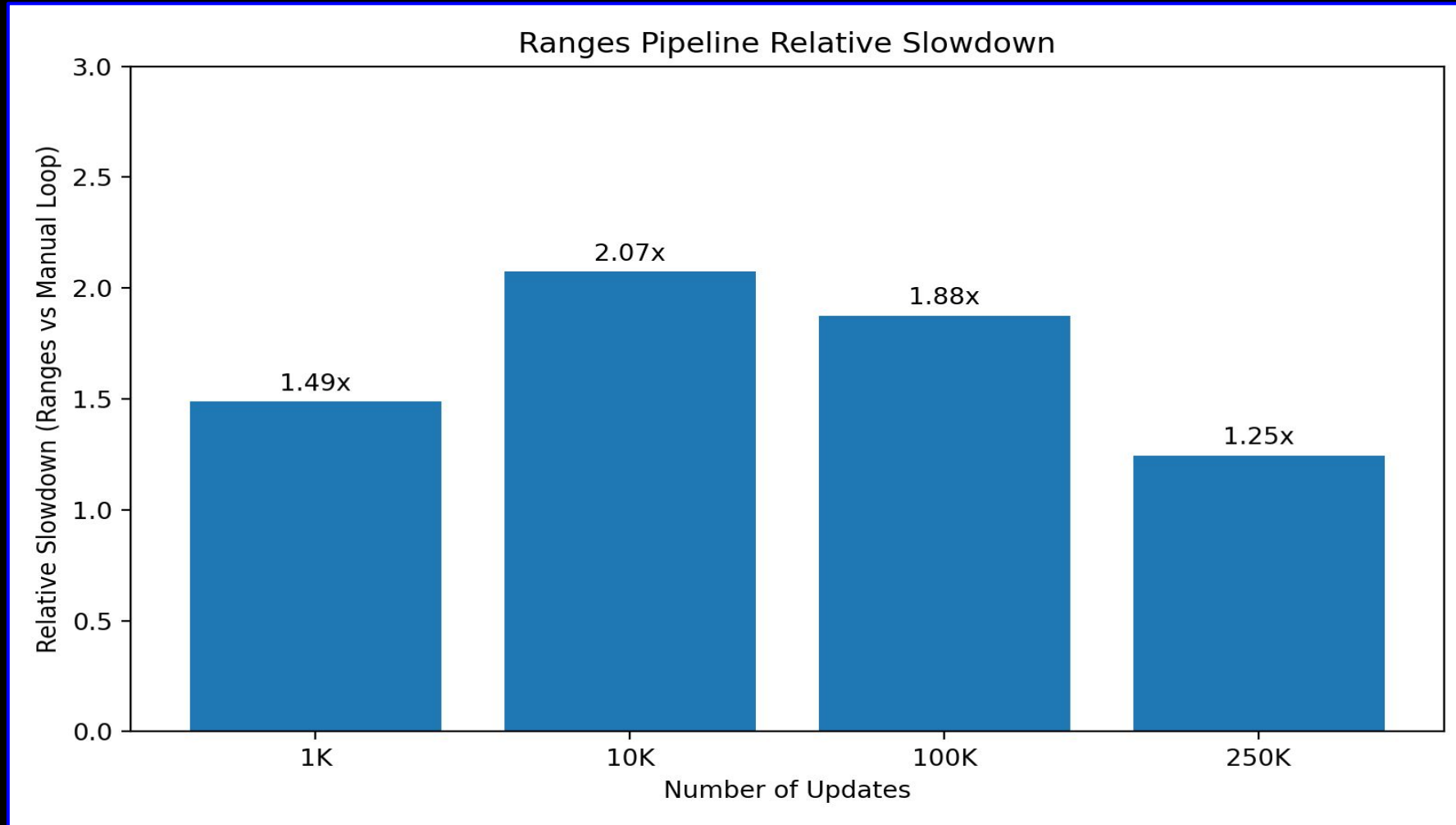
Composition Leaves More Behind

```
cmp    BYTE PTR [rax+0x10],0x0
je
...
mulsd  xmm2,xmm1
comisd xmm4,xmm2
jae
...
jae
addsd  xmm0,xmm1
add    rax,0x18
cmp    rdx,rax
jne ...
```

```
cmp    BYTE PTR [rax+0x10],0x0
jne
...
add    rax,0x18
cmp    rdx,rax
jne
...
movsd  xmm0,QWORD PTR [rax]
mulsd  xmm0,QWORD PTR [rax+0x8]
comisd xmm0,xmm1
jbe
...
```

- Manual loop: One compact control-flow structure
- Ranges pipeline: Filter/iterator advancement logic remains

Measuring Composition Costs



Modular Market Data Processing

- Market data processing split into reusable pipelines
- Each stage owns and materializes its intermediate results
- Improves modularity, testability and stage-level reuse
- Individual stages can be evolved without rewriting the pipeline



Fused Processing Loop

```
constexpr double min_notional = 1'000'000.0;
constexpr double min_adjusted_price = 100.0;
constexpr double adjustment_factor = 1.01;
double sum = 0.0;

for (const auto& update : updates) {
    if (!update.active)
        continue;

    if (update.price * update.quantity <= min_notional)
        continue;

    double adjusted_price = update.price * adjustment_factor;

    if (adjusted_price <= min_adjusted_price)
        continue;

    sum += adjusted_price;
}
```

- Processing performed in a single pass
- No intermediate containers or ownership boundaries
- Minimizes allocation and data movement

Reusable Processing Stages

```
auto active = filter_active(updates);

auto adjusted =
    apply_adjustment(active);

auto filtered =
    filter_large_adjusted(adjusted);

double sum = 0.0;

for (const auto& update : filtered) {
    sum += update.adjusted_price;
}
```

- Processing decomposed into independent reusable stages
- Intermediate results become materialized containers
- Ownership and lifetime boundaries are explicit

Materialization Becomes Runtime Work

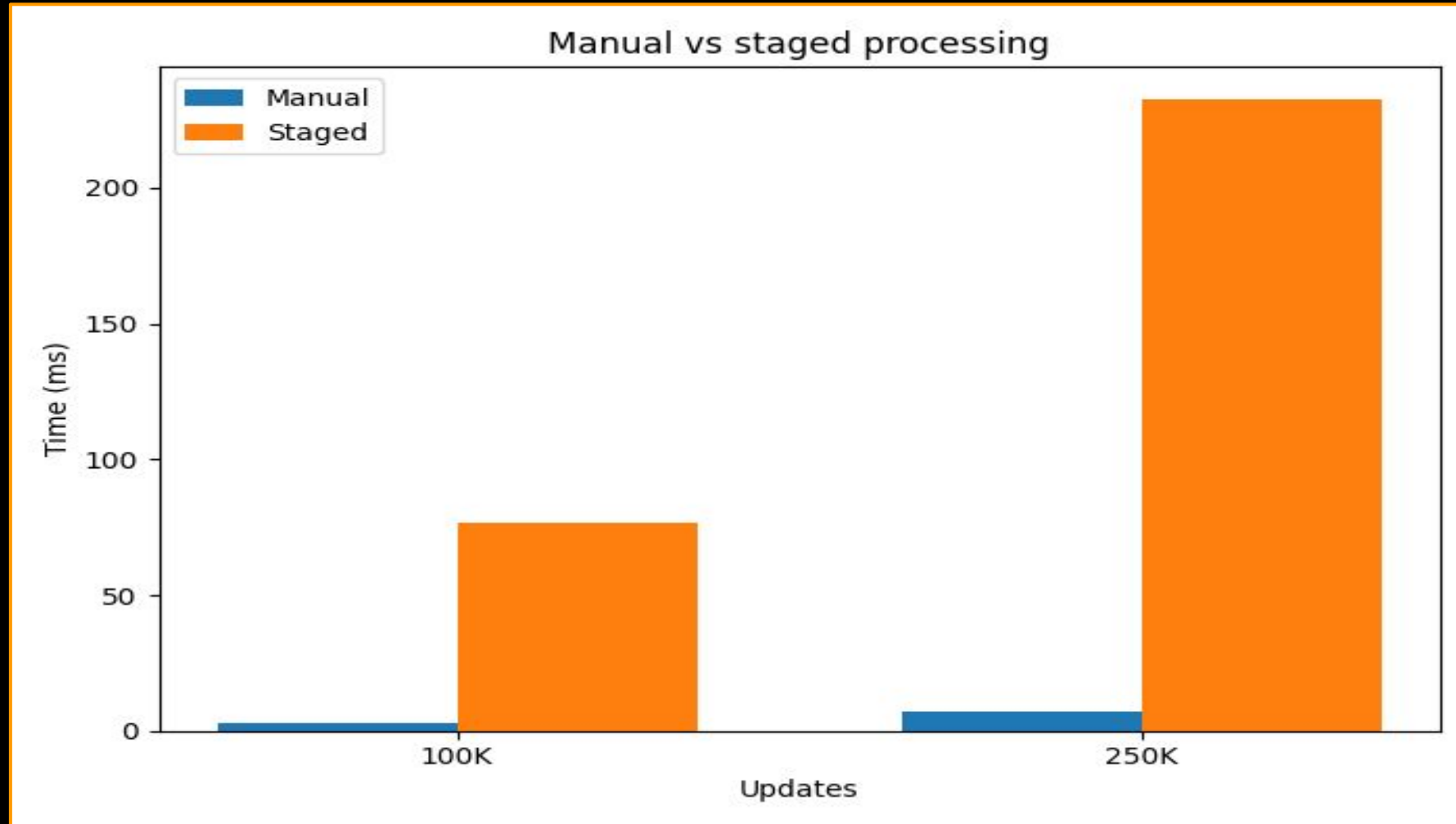
```
cmp    BYTE PTR [rax+0x10],0x0
je
...
mulsd  xmm1,xmm0
comisd xmm3,xmm1
jae
...
addsd  xmm2,xmm0
...
add    rax,0x18
cmp    rdx,rax
jne ...
```

```
call   filter_active
call   apply_adjustment
call   filter_large_adjusted

call   operator new
call   memmove
call   operator delete
```

- Manual version: Pipeline is fused into one loop
- Staged version: Stage boundaries remain

Materialization Costs Dominate





Where Abstraction Costs Come From

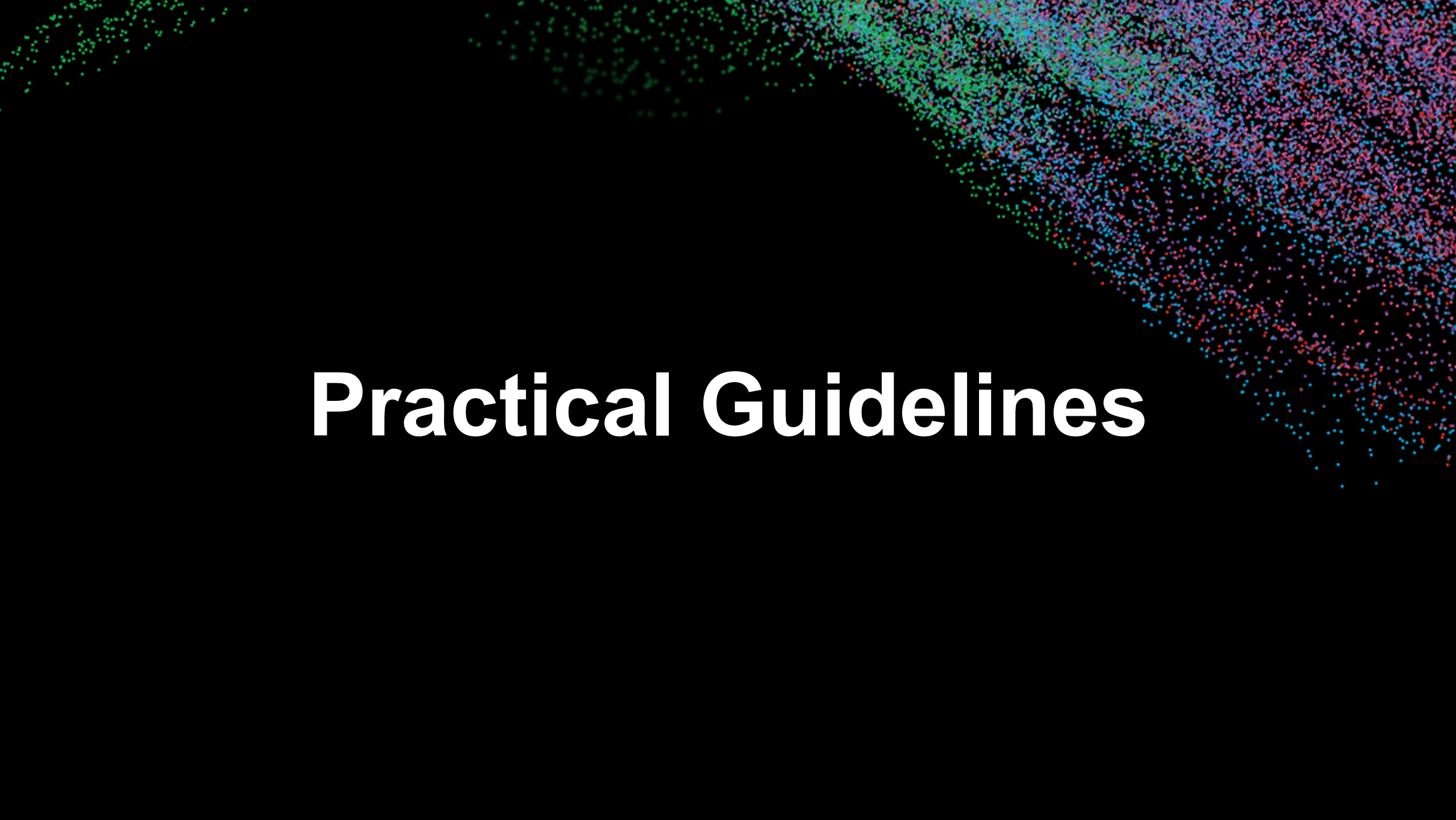
Different Sources of Abstraction Cost

Example	Why costs survived
<code>std::function</code>	Runtime flexibility/erased information
Ranges/Views	Composition complexity
Staged Containers	Ownership and materialization semantics

Common Patterns Behind Abstraction Costs

- Information hidden from the compiler
- Runtime indirection and dynamic behavior
- Deeply layered composition
- Additional ownership and lifetime semantics
- Allocation and materialization





Practical Guidelines

Preserve Optimization Opportunities

- Prefer static polymorphism when runtime flexibility is unnecessary
- Avoid unnecessary indirection in hot paths
- Keep critical processing visible and localized
- Fuse operations where practical



Why Not Just Use -O3?

- Higher optimization levels can help - sometimes substantially
- But they cannot recover information the abstraction erased
- And they cannot remove work the semantics require
- And they are not free:
 - ◆ Larger code and more instruction cache pressure
 - ◆ Longer compile times
 - ◆ Occasionally slower than -O2



Be Intentional About Runtime Behavior

- Modular abstractions can improve clarity and reuse
- Materialization and ownership introduce real runtime work
- Reuse storage and avoid unnecessary allocation where practical
- Benchmark abstractions under realistic workloads



Key Takeaways

- Abstractions provide enormous engineering value
- Different abstractions introduce different kinds of costs
- Some costs can disappear during optimization, others cannot
- Performance requires understanding the runtime implications of the abstractions we use



Questions?

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Career Opportunities



Learn more about our work



Engineering

Bloomberg

TechAtBloomberg.com